

Refinamento de testes com GenAI - ServeRest API

Autor	Gustavo Santos Queiroz
Squad	01
Trilha	AWS & AI for Software Quality Engineering - Nov/2025 - Fase 2
Modelo de AI usada	Gemini 3.1 Pro

PLANO DE TESTES INICIAL

ID	Título do Cenário	Tipo	Endpoint	Dados de Entrada	Resultado Esperado	Status Code
CT01	Cadastrar Usuário Administrador	Funcional (Positivo)	POST /usuarios	Nome, Email dinâmico, Senha, administrador: "true".	Usuário criado com sucesso. Retorna mensagem e _id gerado.	201 Created
CT02	Realizar Login (Obter Token)	Funcional (Positivo)	POST /login	Email e Senha válidos utilizados no CT01.	Autenticação validada com sucesso. Retorna o Token Bearer (salvo em	200 OK

					variável) .	
CT03	Cadastrar Produto no Estoque	Funcional (Positivo)	POST /produtos	Header: Token (CT02). Body: Nome único, Preço válido, Descrição, Qtd.	Produto registrado no sistema. Retorna mensagem de sucesso e o <code>_id</code> do produto.	201 Created
CT04	Fluxo de Carrinho (Adicionar e Concluir)	Funcional (Positivo)	POST /carrinhos e DELETE E	Header: Token (CT02). Body: <code>idProduto</code> (CT03) e Quantidade (1).	Item adicionado e compra concluída. Valida a limpeza do carrinho com mensagem de exclusão.	201 e 200
CT05	Tentativa de Login com Senha Inválida	Funcional (Negativo)	POST /login	Email válido (CT01), porém com senha incorreta (ex:	Acesso bloqueado. A API retorna a mensagem	401 Unauthorized

				"senha_errada").	exata: "Email e/ou senha inválidos".	
CT06	Cadastrar Produto com Preço Negativo	Funcional (Negativo)	POST /produtos	Header: Token Admin. Body com campo de preço inválido (preço: -50).	Cadastro rejeitado por violação de regra de negócio. Retorna mensagem exigindo preço positivo.	400 Bad Request
CT07	Validar Contrato de Lista de Usuários	Contrato	GET /usuarios	Nenhum (parâmetros de busca vazios).	Retorna JSON válido com as chaves estruturais obrigatórias: quantidade, nome, email, password, admini	200 OK

					strado r e _id.	
CT08	Validar Cadastro de Produto com Preço Decimal	Regress ão (Bug- 01)	POST /produ tos	Header: Token Admin. Body com preco: 50.99.	(Falha Esperad a) O sistema deveria aceitar decimai s, mas rejeita com erro.	201 (Retorna 400)
CT09	Validar Rejeição com E-mail Inválido	Regress ão (Bug- 02)	POST /usuar ios	Body com e- mail inválido contend o caracter es especiai s/emojis (#emai linval ido🚀 @qa.co m).	(Falha Esperad a) O sistema deveria rejeitar o formato, mas aceita indevida mente, criando o usuário.	400 (Retorna 201)

PROMPT

Atue como um Engenheiro de Qualidade Sênior (QA) e Arquiteto de Testes, especialista em design, estratégia e modelagem de testes para APIs REST.

Contexto: Desenvolvi um Plano de Testes documentado para os endpoints da API pública ServeRest (um simulador de e-commerce). O meu plano atual é composto por 9 cenários e já cobre: Caminho Feliz, Caminhos de Erro, Validação de Contrato e Testes de Regressão de Bugs.

Tarefa: Realize um **Refinamento Crítico focado exclusivamente no planejamento dos testes**. Analise a tabela de cenários que fornecerei em anexo e identifique lacunas na minha cobertura. O foco é 100% na estratégia e no refinamento dos casos de teste.

Restrições e Foco da Análise: Analise a minha tabela e sugira novos Casos de Teste (CTs) baseados nas seguintes categorias de risco que podem ter ficado de fora:

1. **Testes de Segurança e Autorização:** Ex: Tentar acessar rotas de administrador com token de utilizador comum, ou usar tokens expirados/inválidos.
2. **Edge Cases e Valores Limite:** Ex: Comportamento da API ao enviar campos obrigatórios vazios, nulos, ou com strings excessivamente longas.
3. **Resiliência e Formato de Dados:** Ex: Enviar payload em XML em vez de JSON, ou enviar tipos de dados incorretos (string num campo de inteiro, além do erro de float que já mapeei).
4. **Concorrência/Regras de Negócio Ocultas:** Ex: Tentar cadastrar o mesmo e-mail duas vezes, ou deletar um produto que já está no carrinho de um utilizador.

Formato de Saída Esperado: Entregue o resultado estruturado na forma de um "**Relatório de Refinamento de Plano de Testes**", contendo:

- Uma breve análise crítica geral do plano atual.
- Um **Checklist de Melhorias e Novos Cenários Elicitados** (categorizados de forma clara, ex: Segurança, Edge Cases, Performance).
- O formato deve ser altamente profissional, estruturado em tópicos.

O plano de testes está em anexo.

RESPOSTA

Relatório de Refinamento de Plano de Testes: API ServeRest

1. Análise Crítica Geral do Plano Atual

O seu plano de testes atual apresenta uma fundação sólida e bem estruturada. A cobertura de 9 cenários demonstra uma visão madura da qualidade do software, especialmente por já contemplar não apenas o "Caminho Feliz" (CT01 a CT04), mas também cenários negativos críticos (CT05, CT06), validação de contrato (CT07) e uma mentalidade preventiva com a regressão de bugs conhecidos (CT08, CT09).

No entanto, para elevarmos este material ao padrão de uma arquitetura de testes sênior e garantirmos a robustez de uma aplicação de e-commerce em ambiente de produção, precisamos expandir a matriz de cobertura. A API ServeRest possui regras de negócio implícitas e comportamentos não-determinísticos que não estão totalmente cobertos. O foco agora é "estressar" a aplicação com validações rigorosas de segurança, concorrência e quebra de contratos de formatação de dados.

2. Checklist de Melhorias e Novos Cenários Elicitados

Abaixo, detalho os novos Casos de Teste (CTs) que devem ser incorporados ao seu framework, categorizados por áreas de risco:

2.1. Segurança e Autorização

A sua suíte já valida o bloqueio de login com credenciais incorretas (CT05), mas precisamos testar as barreiras de autorização baseadas em perfil de usuário e integridade do token.

- **CT10 - Tentativa de Acesso Privilegiado com Usuário Comum:** Tentar realizar uma ação administrativa, como o cadastro de produto (POST `/produtos`), utilizando um token gerado por um usuário comum (onde a flag `administrador` seja `"false"`). *Resultado Esperado:* Retorno `403 Forbidden` com mensagem de rota exclusiva para administradores.
- **CT11 - Acesso com Token Malformado ou Ausente:** Tentar acessar o fluxo de carrinho (POST `/carrinhos`) omitindo completamente o *Header* de `Authorization`, ou enviando uma string aleatória (ex: `Bearer 12345`). *Resultado Esperado:* Retorno `401 Unauthorized`.
- **CT12 - Comportamento com Token Expirado:** Simular (se a arquitetura de testes permitir) o envio de uma requisição para uma rota protegida utilizando um token JWT cujo tempo de vida útil (TTL) já tenha expirado. *Resultado Esperado:* Retorno `401 Unauthorized`.

2.2. Concorrência e Regras de Negócio Ocultas

O seu fluxo de carrinho (CT04) e de cadastro (CT01, CT03) validam a inserção inicial. Precisamos testar como a API lida com conflitos de estado e dados duplicados.

- **CT13 - Tentativa de Cadastro Duplicado:** Tentar enviar o POST `/usuarios` com um e-mail que já foi registrado com sucesso em um teste anterior, ou cadastrar um produto (POST `/produtos`) com um nome já existente. *Resultado Esperado:* Retorno `400 Bad Request` indicando que o e-mail/nome já está em uso.
- **CT14 - Exclusão de Entidade em Uso:** Tentar enviar um DELETE para um produto (criado no CT03) enquanto ele ainda consta ativamente no carrinho de um usuário (antes do fluxo de conclusão do CT04). *Resultado Esperado:* Retorno de erro (`400` ou `409 Conflict`) informando violação de chave estrangeira ou regra de negócio.
- **CT15 - Violação de Limite de Estoque:** No fluxo de carrinho (POST `/carrinhos`), tentar adicionar uma quantidade de produto que exceda a quantidade cadastrada inicialmente no estoque (CT03). *Resultado Esperado:* Retorno `400 Bad Request` indicando estoque insuficiente.

2.3. Edge Cases e Valores Limite (Boundary Values)

Já mapeamos o preço negativo (CT06), mas as validações de input precisam garantir a integridade da base de dados contra anomalias.

- **CT16 - Omissão de Campos Obrigatórios (Missing Fields):** Enviar o *Body* do POST `/usuarios` ou POST `/produtos` omitindo chaves fundamentais, como `email` ou `nome`. *Resultado Esperado:* Retorno `400 Bad Request` com a lista de campos faltantes.
- **CT17 - Estouro de Limite de Caracteres:** Enviar payloads contendo strings excessivamente longas (ex: mais de 255 caracteres) nos campos de `nome` ou `senha` (CT01). *Resultado Esperado:* Tratamento adequado do erro (geralmente `400`), sem estourar `500 Internal Server Error`.
- **CT18 - Quantidade Nula ou Negativa no Carrinho:** Tentar adicionar um produto ao carrinho (POST `/carrinhos`) passando o valor `0` ou `-1` no campo de quantidade. *Resultado Esperado:* Retorno `400 Bad Request`.

2.4. Resiliência e Formato de Dados

Já possuímos casos de regressão de formatação como o bug de e-mail (CT09) e tipos decimais (CT08). Precisamos validar a camada de transporte.

- **CT19 - Payload com Content-Type Incorreto:** Enviar a requisição de autenticação (POST `/login`) formatando o *Body* em XML ou *Form-Data*, mas informando (ou não) um *Content-Type* incompatível. *Resultado Esperado:* Retorno `415 Unsupported Media Type` ou `400 Bad Request`.

- **CT20 - Tipagem Rígida e Type Casting Incorreto:** Enviar valores numéricos nos campos que esperam string (ex: "nome": 12345 no CT01), ou enviar booleanos no lugar de inteiros (ex: "quantidade": true no CT03). *Resultado Esperado:* A API deve rejeitar a requisição estruturalmente (400 Bad Request) antes de tentar processar a regra de negócio.